Swarm Intelligence

Report on Assignment 3

Aaron Bussche David Magaram Jason Kasdiran

Group 28

March 9, 2026

1 Part A

1.1 Introduction

An image can be compressed by reducing the number of distinct colors in it. This process is called color quantization. The objective of finding the optimal color palette for a given image can be seen as a clustering problem. In this assignment, we tackle the color quantization problem using Particle Swarm Optimization (PSO). We implement standard PSO to iteratively optimize image color palettes and then evaluate its performance by comparing the resulting quantization against a popular clustering method, K-Means.

1.2 Preliminary Concepts

To process the image digitally, a color is represented numerically as a 3-dimensional vector corresponding to its Red, Green, and Blue (RGB) channel intensities, and a target color palette is represented as a 2D matrix of shape $(N, 3)$, where N is the number of available colors in the palette.

To quantize an image using a given palette, we implemented a custom function to evaluate every pixel in the input image array. For each pixel, it computes the squared Euclidean distance between the pixel's RGB vector and each of the N candidate color vectors in the palette. The original pixel is then replaced by the closest color in the given palette. Testing this function on the provided image and our own image using the pre-defined 4-color palette successfully reproduced the correct target quantization (Appendix 1).

To find an optimal color palette for an image we appraoched it as a clustering problem and used the popular K-Means clustering algorithm. In this clustering context:

- The **data** points represent the individual pixels of the original image, defined by its (R, G, B) values.
- The clusters represent groupings of pixels with similar RGB values. The **centroid** of each cluster acts as the representative color for all pixels assigned to that cluster. Therefore, the list of the N optimal cluster centroids directly forms the optimal N-color palette.

By clustering the pixels, the goal is to minimize the intra-cluster variance, which in this case equates to minimizing the Mean Squared Error (MSE) between the original image pixels and the restricted colors of the new palette.

1.3 Methods

To map the color quantization problem to PSO, each particle in the swarm represents a candidate solution, which in this case is a list of N cluster centroids. Mathematically, a particle's position is defined as $x_i = (m_{i1}, \ldots, m_{iN})$, where each centroid m corresponds to a color in the palette.

We measure the quality or "fitness" of a particle by computing an evaluation function $f(x_i, Z)$ over the image dataset Z containing M pixel points. During the evaluation, each pixel z_j is assigned to the nearest centroid m. The implementation maps the input image to the color palette by finding the minimum Euclidean distance between each pixel and the candidate colors, outputting the reconstructed

image to calculate the Mean Squared Error (MSE). The PSO algorithm stores the state of the particles and the global best particle at each iteration, allowing the swarm to minimize the MSE over time.

1.4 Experimental Setup

The PSO clustering algorithm was implemented and evaluated on three test images: "peppers", "colorful", and "very small". Because the PSO algorithm can take very long to evaluate fitness on large datasets, the "colorful" image was resized from 1328×2000 to a smaller resolution of 85×128 prior to running the optimizer.

The experimental parameters were fixed for all test cases. The target color palette size was set to $N = 4$ colors. The PSO swarm consisted of 30 particles, and the algorithm was executed for a maximum of 100 iterations. To benchmark our implementation, the standard K-Means algorithm (via `sklearn.cluster.KMeans`) was applied to the same images with $K = 4$. Both algorithms were quantitatively compared based on the final MSE of the quantized image.

1.5 Results

The PSO implementation successfully quantized all three images to a 4-color palette and optimization curves were generated to show the convergence over 100 iterations. The final Mean Squared Error (MSE) values comparing PSO and K-Means are summarized below:

- **Peppers:** PSO achieved a final MSE of 524.25, slightly outperforming K-Means, which achieved an MSE of 524.26.
- **colorful:** K-Means performed better with an MSE of 1358.31, while PSO settled at a final MSE of 1373.19.
- **Very Small:** PSO significantly outperformed K-Means, reaching a final MSE of 4324.48 compared to the K-Means MSE of 4565.67.

The historical states of the global best particle showed that PSO achieved the most dramatic reductions in MSE during the first 30 to 40 iterations, after which the curves converges.

1.6 Discussion

The results confirm that Particle Swarm Optimization can successfully be applied to color quantization and is capable of finding optimal, or near-optimal, color palettes. On two of the three tested images ("peppers" and "very small"), the PSO method discovered a color palette that performed better than the K-Means baseline. This highlights the advantage of PSO as a global search algorithm; it avoids the heavy reliance on strict initial centroid placements that K-Means suffers from.

However, the experiment also illustrated some known limitations of standard PSO. The fact that K-Means performed better on the "colorful" image demonstrates that PSO can still easily get stuck in local optima during the clustering search space. Additionally, PSO is highly computationally demanding and slow to converge. Evaluating the fitness of 30 particles over 100 iterations requires exhaustively assigning every image pixel to its nearest centroid thousands of times, which justified our need to downscale images to reduce runtime. Ultimately, while PSO is an effective optimization strategy for clustering, simpler alternatives like K-Means are often much faster and sufficiently accurate for typical applications.

2 Part B

2.1 Introduction

The Boids model from Craig Reynolds is one of the most popular models of swarm intelligence and group behavior. It uses boids, which are bird like objects, that follow 3 simple rules: alignment, cohesion, and separation. These simple rules govern the interactions between individual boids, which gives rise to a more complex emergent behaviors such as flocking. This is also known as swarm intelligence. Swarm intelligence is some collective behavior that emerges when individual agents interact

with each other. This can lead to flocking, fish schools, and insect swarms. The goal of this project is to implement the Boids model and explore how the swarm behavior emerges from simple rules and how modifying the parameters affect it.

2.2 Methods

The JavaScript framework was used to implement the two-dimensional Boids model. The model was defined on a square grid of 400x400 pixels with wrapped dimensions. 150 boids were randomly initialized, each moving at a constant speed. The position and direction of each boid were updated at each time step based on the three rules, while the current boid was not included in the calculations. The simulation was ran for 300 time steps and the used parameters in this project are displayed in Table 1 below.

Boids follow three simple interaction rules: alignment, cohesion, and separation. Alignment controls how each boid moves towards the average direction of the other boids nearby. Similarly, cohesion controls how quickly each boid tries to move towards the average position of the other boids around. Separation dictates how much boids try to keep a certain distance from each other to avoid collision. Together, these rules dictate how boids interact with each other. However, other than these 3 basic behavior rules, each boid has an inner and outer radius. The outer radius is associated with cohesion and alignment, while the inner radius is linked to separation. The outer radius is the boids' visual and interaction range. It is the distance within which the boid can see other boids and interact with them, and is visualized in Figure 1 as a gray circle surrounding the boid. The inner radius, depicted by a red circle around the boid shown in Figure 1, prevents collision and is the opposite of the interaction range. If a boid enters another boids inner radius, they are repelled away, controlled by the separation weight. For obvious reasons, the outer radius should be larger than the inner radius so cohesion and alignment can occur.

Parameter	**Value**
$Radius_{\text{inner}}$	10
$Radius_{\text{outer}}$	25
Alignment weight	1
Coherence weight	1
Separation weight	1
Number of boids	150

Table 1: Parameters.

To analyze the swarm, the order parameter and nearest-neighbor distance quantitative metrics were used. Order parameter is the average normalized velocity of the Boids. Order parameter tells us to what degree the swarms align in their directions, and the output is squashed between 0 and 1, 0 meaning completely random directions and 1 meaning perfect alignment.

$$O = \frac{1}{N_B} \left\| \sum_{i=1}^{N_B} \frac{\vec{v}_i}{\|\vec{v}_i\|} \right\|$$

The nearest-neighbor distance is the average nearest-neighbor distance across all boids. It measures the total average distance across all boids and can tell us when individual boids get separated from the swarm or when individuals get too close to each other.

$$d_i = \min_{\substack{j=1 \\ j \neq i}}^{N_B} \|\mathbf{x}_i - \mathbf{x}_j\|$$

$$\bar{d} = \frac{1}{N_B} \sum_{i=1}^{N_B} d_i = \frac{1}{N_B} \sum_{i=1}^{N_B} \min_{\substack{j=1 \\ j \neq i}}^{N_B} \|\mathbf{x}_i - \mathbf{x}_j\|$$

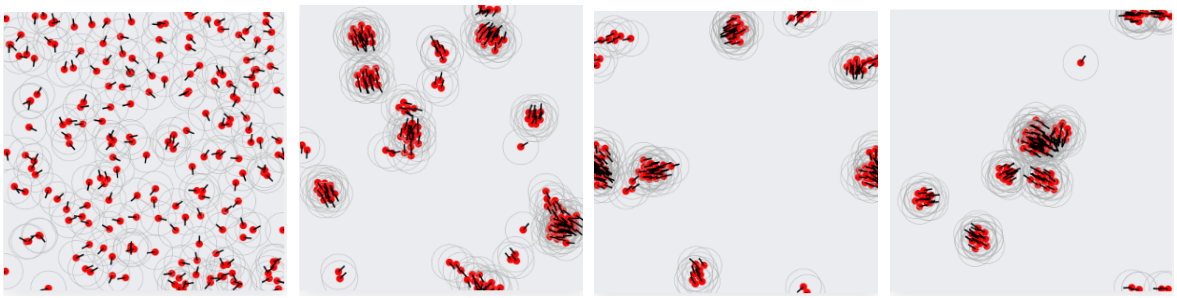
2.3 Results

The visualizations in Figure 1 show the positions of the boids at different time steps. Four snapshots were taken at time steps 0, 100, 200, and 300, and are separated into an initial state, final state, and two intermediate states. Red color represents the inner radius, while the black bar coming out of it indicates in which direction each boid is facing. The outer radius is shown as a gray circle and is each boids' interaction range.

150 boids were initialized randomly on a 400x400 field with wrapped boundaries. Boids then move around, updating their positions and orientations based on the alignment, coherence, and separation rules. Boids interact with each other by coming in each other interaction range, forming flocks as they align and cohere with each other, while also maintaining a certain distance to avoid collisions. Over time, larger flocks form as boids and smaller flocks join together. However, as shown in Figure 1d, not all boids are able to find a flock by time step 300 and remain isolated until they bump into another boid or flock.

The boids are distributed across the simulation field, and their positions and orientations change over time as they interact with each other based on the alignment, coherence, and separation rules.

Over time, we can observe the formation of flocks as the boids align and cohere with each other. The separation behavior can also be seen as boids maintain a certain distance from each other to avoid collisions.



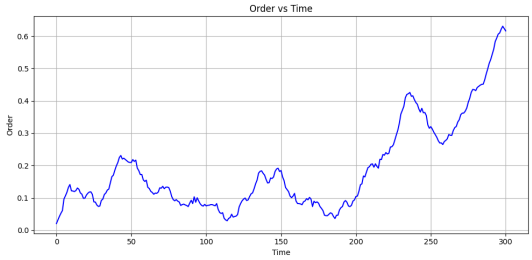
(a) Time: 0 steps (b) Time: 100 steps (c) Time: 200 steps (d) Time: 300 steps

Figure 1: Visualizations of boids at four different time steps using default parameters.

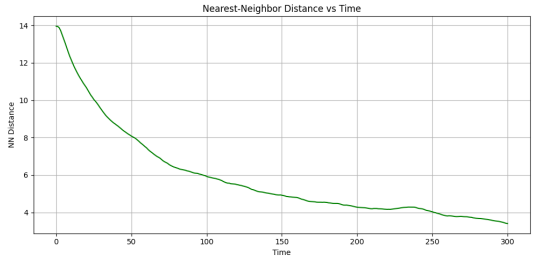
Table 2 accompanies the visualizations from Figure 1 and displays the order parameter and nearest-neighbor distance values obtained at the initial, intermediate, and final states. For further analysis, Figure 2 plots display the order parameter and nearest-neighbor distance values against time. A pattern can be observed where order parameter grows and nearest-neighbor distance shrinks as time steps increase. This is expected as smaller flocks that formed during the initial phase wander around until they bump into each other, merging and creating a large unified flock.

Time	**0**	**100**	**200**	**300**
Order parameter	0.0208	0.0753	0.105	0.616
Nearest-neighbor distance	13.956	5.925	4.278	3.405

Table 2: Quantitative results for default parameters.



(a) Order parameter over time



(b) Nearest-neighbor distance over time

Figure 2: Plots of quantitative results over time.

A parameter sweep was conducted with the parameter configurations displayed in Table 3. In total, 384 different configurations were explored and the top 32 were tested further to help with noise. Two top configurations were chosen, one with the highest order parameter, and another with the highest NN-distance and ratio. The parameters the top 2 configurations used are displayed in Table 4. From the results in Table 4 it appears that to maximize order, a large outer radius and high alignment weight are important, while cohesion should be disabled. To minimize nearest-neighbor distance, both alignment and cohesion should be set to their maximum values. The best NN-distance and best order/NN ratio are achieved at the same configuration (align=1, cohesion=1, outerR=40), which maximizes NN distance while having mediocre order results. All best results had separation=0.

Parameter	**Value**
$Radius_{\text{inner}}$	5, 10, 20
$Radius_{\text{outer}}$	25, 40
Alignment weight	0, 0.33, 0.66, 1
Coherence weight	0, 0.33, 0.66, 1
Separation weight	0, 0.33, 0.66, 1
Number of boids	150

Table 3: Explored parameter configurations, 384 total. Each were run 3 times with the results averaged. In Table 4, the most promising 32 were tested more thoroughly to combat noise.

Criterion	R_{out}	**Align**	**Cohes.**	**Order**	**NN-dist**
Best Order	40	1	0	0.991	13.11
Best NN-dist & Ratio	40	1	1	0.446	0.10

Table 4: Parameter configurations that maximize each criterion at $t = 300$, selected from 32 configurations (N=150, averaged over 25 repeats). Order and NN-dist columns show the resulting metrics. All best results had separation=0, rendering R_{in} irrelevant, so for better clarity these parameters are omitted from the table.

2.4 Discussion

The results show that complex swarm behavior can emerge from simple interaction rules. We observed that the boids flock quite well even at the default parameters. The order parameter and NN-distance usually only get really good at the 'end stage' when there is just one combined flock of all the boids. How fast this end stage is achieved (and how tightly the boids are packed in the flock and how perfectly aligned they are) depends on the parameter choices, but for most sensible parameter choices this eventually happens. Examples for which this does not hold would be having 0 alignment and cohesion or having a high separation and an $R_{\text{in}} > R_{\text{out}}$. There likely is a specific threshold plane that divides convergent behavior from divergent one, but we did not test this out in detail.

By conducting the parameter sweep, we found the best parameters for how to maximize the Order and minimize the NN-distance, which were consistently extreme values (R_{out} as big as possible,

alignment=1, separation=0, etc.). This made us question the purpose of maximizing for these results in the first place: If we want a model with all boids in the same exact place and facing the exact same direction, that is not how nature works, and nature is what we are trying to model. It therefore might be more interesting to find out what parameters perfectly mirror bird behavior.

Other than that, several interesting noteworthy behaviors were observed during the exploration of different parameter configurations. For example, when a low alignment weight with medium separation weight were used, it caused the boids to continuously spin around. Also, low separation weight causes the boids to enter each other, becoming one. Additionally, increasing the outer radius gives high cohesion and very short time because boids have a much larger interaction rate. On the other hand, the inner radius should generally be smaller than the outer radius for interactions to happen. A rule of thumb is to have the outer radius be at least twice the size of the inner radius.

We are confident that this project's code implementation is correct as adjusting the parameters gives the desired results, however, it is possible that some bugs and artifacts remain that corrupting our findings. Additionally, it is important to note that due to the random initializations the results can greatly vary between runs. It is recommended that future studies make sure the code implementation is correct and without any issues. Also, averaging the obtained quantitative metric scores across many runs is advised for more stable results.

2.5 Conclusion

In this experiment, the Boids model for swarm intelligence was implemented and analyzed both visually and quantitatively. The results show that alignment increases over time and it is possible to reach high alignment within 300 time steps. Moreover, the decrease of nearest-neighbor distance indicates that boids form flocks that gradually increase in size.

This experiment reproduced the Boids model for swarm intelligence and demonstrated that simple interaction rules can give rise to complex emergent behavior.